Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

COURSE CODE: DJS23ILPC301 **DATE:** 15/10/25

COURSE NAME: Data Structure Laboratory **CLASS:** IT-1

NAME: Varun Nembhani **SAP ID:** 60003240204

Experiment No. 9

CO/LO:

Aim: Implement Linear & Binary Search, Fibonacci Search.

Theory:

Department of Information Technology

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

DATE: 14/10/23

COURSE CODE: DJS23ILPC301

CLASS: I1 - 3

COURSE NAME: Data Structure Laboratory

SAP ID: 60003240204

NAME: Varun Nambiar

Experiment No. 4.

28/10

CO/LO: LO1, LO2

Aim: Implement linear and binary search, Fibonacci Search

Theory:

1) Linear Search - It is the simplest technique that checks each element in the list one by one until element is found or list ends.

Algo: LinearSearch (arr, n, key)
for (int i = 0 to n-1)
  if (arr[i] == key then
    return 1
  end if
end for
Return n-1

2) ~~Its~~ Binary search - Its ~~an~~ used on sorted arrays. It repeatedly divides the search range in half to locate the target element.

Algo: Binary-Search (arr, n, key)
low = 0
high = n-1
while (low <= high) do
  mid = (low + high) / 2
  if arr[mid] == key then
    return mid
  else if arr[mid] > key then
    high = mid - 1
  else
    low = mid + 1
end while
return - 1

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)
Department of Information Technology

Fibonacci search:
and Algo: It's a comparison based search algorithm that works on sorted arrays. It uses fibonacci series to narrow search interval and it is often more efficient than Binary search in certain cases.

Algorithm: Fib-Search(arr, n, key)

```
    ~~Set~~ F1 = 0
            F2 = 1
  Fm = F1 + F2
  while Fm < n do
     F1 = F2
     F2 = Fm
     Fm = F1 + F2
endwhile
 offset = -1
 while Fm > 1 do
  i = min(offset + F1, n-1)
  if arr[i] < key then
     Fm = F2
     F2 = F1
     F1 = Fm - F2
     offset = i
  else if arr[i] > key then
     Fm = F1
     F2 = Fm - F1
     F1 = Fm - F2
  else
     return i
endwhile
if F2 && arr[offset + 1] == key
   return offset + 1
end if
```

Conclusion:

Linear Search is the simple technique that work on unsorted data but slower for larger dataset.
Binary Search is efficient with a time complexity of O(logn), but it requires the data to be sorted.
Fibonacci works on sorted array and uses fibonacci numbers to narrow the search range.

CODE:

```
#include <stdio.h>

void sortArray(int arr[], int n) {
    int temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int key) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}

int fibonacciSearch(int arr[], int n, int key) {
    int fibMMm2 = 0;
    int fibMMm1 = 1;
    int fibM = fibMMm2 + fibMMm1;

    while (fibM < n) {
        fibMMm2 = fibMMm1;
```

```
        fibMMm1 = fibM;
        fibM = fibMMm2 + fibMMm1;
    }

    int offset = -1;
    while (fibM > 1) {
        int i = (offset + fibMMm2 < n - 1) ? offset + fibMMm2 : n - 1;

        if (arr[i] < key) {
            fibM = fibMMm1;
            fibMMm1 = fibMMm2;
            fibMMm2 = fibM - fibMMm1;
            offset = i;
        }
        else if (arr[i] > key) {
            fibM = fibMMm2;
            fibMMm1 = fibMMm1 - fibMMm2;
            fibMMm2 = fibM - fibMMm1;
        }
        else
            return i;
    }

    if (fibMMm1 && arr[offset + 1] == key)
        return offset + 1;

    return -1;
}

int main() {
    int n, key, choice, cont;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\nEnter the key to search: ");
        scanf("%d", &key);

        printf("\nChoose search method:\n");
        printf("1. Linear Search\n");
```

```
        printf("2. Binary Search\n");
        printf("3. Fibonacci Search\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        int result = -1;

        switch (choice) {
            case 1:
                result = linearSearch(arr, n, key);
                break;
            case 2:
                sortArray(arr, n);
                result = binarySearch(arr, n, key);
                break;
            case 3:
                sortArray(arr, n);
                result = fibonacciSearch(arr, n, key);
                break;
            default:
                printf("Invalid choice!\n");
                break;
        }

        if (result != -1)
            printf("Element found at index %d\n", result);
        else
            printf("Element not found.\n");

        printf("\nDo you want to search again? (1 = Yes / 0 = No): ");
        scanf("%d", &cont);

    } while (cont == 1);

    printf("\nProgram terminated.\n");
    return 0;
}
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                C/C++ Comp

Enter 4 elements:
2
5
7
8

Enter the key to search: 2

Choose search method:
1. Linear Search
2. Binary Search
3. Fibonacci Search
Enter your choice: 1
Element found at index 0

Do you want to search again? (1 = Yes / 0 = No): 1

Enter the key to search: 7

Choose search method:
1. Linear Search
2. Binary Search
3. Fibonacci Search
Enter your choice: 2
Element found at index 2

Do you want to search again? (1 = Yes / 0 = No): 1

Enter the key to search: 8

Choose search method:
1. Linear Search
2. Binary Search
3. Fibonacci Search
Enter your choice: 3
Element found at index 3

Do you want to search again? (1 = Yes / 0 = No): 0

Program terminated.
```

HACKEREARTH :

Enter your code or Upload your code as file. Save C (gcc 10.3)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int *seen = (int *)calloc(N + 2, sizeof(int));  // we only need up to N+1
    int mex = 0;

    for (int i = 0; i < N; ++i) {
        int x;
        scanf("%d", &x);

        if (x <= N + 1) seen[x]++;

        while (seen[mex] > 0) mex++;

        if (i) putchar(' ');
        printf("%d", mex);
    }
    putchar('\n');

    free(seen);
    return 0;
}

```

27:1 vscode

Test against custom input ▾ Compile & Test code Submit code

Department of Information Technology

Enter your code or Upload your code as file. Save C (gcc 10.3)

```c
int main(void) {
    int N, M;
    if (scanf("%d %d", &N, &M) != 2) return 0;

    long long INF = (long long)9e18;
    long long max_row_min = -INF;

    // Track column minimums while reading; no need to store the whole matrix
    long long cmin[M];
    for (int j = 0; j < M; ++j) cmin[j] = INF;

    for (int i = 0; i < N; ++i) {
        long long row_min = INF;
        for (int j = 0; j < M; ++j) {
            long long x;
            scanf("%lld", &x);
            if (x < row_min) row_min = x;
            if (x < cmin[j]) cmin[j] = x;
        }
        if (row_min > max_row_min) max_row_min = row_min;
    }

    long long max_col_min = -INF;
    for (int j = 0; j < M; ++j)
        if (cmin[j] > max_col_min) max_col_min = cmin[j];

    long long answer = (max_row_min < max_col_min) ? max_row_min : max_col_min;
    printf("%lld\n", answer);
    return 0;
}
```

34:1 vscode

Test against custom input ▾ Compile & Test code Submit code

Submission ID: 122283456

RESULT: Accepted Refer judge environment